Implementation of A Search Algorithm for Optimal Pathfinding in Graphs

Title: A* Search Algorithm for Graph Traversal

Group members Name:

Amna Javed 23-bcs-009

Ayesha Saleem 23-bcs-022

Almas shafqat 23-bcs-006

Kanwal noor ul ain 23-bcs-049

Course Name: **Artificial Intelligence**

Submission Date: ***19-12-24***

Contents

1. Introduction to Graph Traversal 3

2. Overview of the A Search Algorithm* 3

3. Problem Definition 3

4. Components of the A Search Algorithm* 3

5. Code Implementation 4

6. Explanation of Code 5

7. Graph and Heuristic Example 5

8. Output Analysis 6

..... 6

9. Advantages of A Search* 6

10. Applications of A Search in Real World* 6

11. Conclusion 6

12. References ..... **Error! Bookmark not defined.**

1. Introduction to Graph Traversal

Graph traversal is an essential problem-solving technique in computer science and artificial intelligence. It involves visiting all the nodes or vertices in a graph systematically. Two popular types of traversal methods are:

- **Breadth-First Search (BFS)**
- **Depth-First Search (DFS)**

However, for finding the **optimal path** in weighted graphs, algorithms like **A*** search and Dijkstra's Algorithm are used.

In this project, we implement the *A Search Algorithm** that utilizes a combination of cost and heuristic values to find the most efficient path from a starting node to a goal node.

*2. Overview of the A Search Algorithm**

The **A*** algorithm is a heuristic search algorithm that finds the shortest path in a graph.
It uses the formula:

f(n)=g(n)+h(n)f(n) = g(n) + h(n)f(n)=g(n)+h(n)

- g(n)g(n)g(n): Cost of the path from the start node to the current node nnn.
- h(n)h(n)h(n): Heuristic estimate of the cost from node nnn to the goal.
- f(n)f(n)f(n): Total estimated cost.

The heuristic function h(n)h(n)h(n) must be **admissible** (never overestimates the actual cost).

3. Problem Definition

The problem is to find the **shortest path** between two nodes in a weighted graph using the A* Search Algorithm.

The graph consists of:

- Nodes A,B,C,D,E,FA, B, C, D, E, FA,B,C,D,E,F
- Weighted edges with associated costs
- A heuristic table estimating the cost from each node to the goal.

The goal is to identify the most optimal path and calculate the total cost.

*Components of the A Search Algorithm**

The A* Search Algorithm involves the following components:

1. **Graph Representation**: Adjacency list to store nodes and edges.
2. **Heuristic Function**: A function to estimate the remaining cost.
3. **Priority Queue**: Stores nodes based on their f(n)f(n)f(n) value.
4. **Path Tracking**: Keeps track of the path and costs.

The algorithm proceeds as:

1. Start from the initial node.
2. Use the heuristic and current path cost to determine the next node.
3. Stop when the goal node is reached.

5. Code Implementation

```
import heapq

class Graph:
    def __init__(self):
        self.graph = {}
        self.h_values = {}

    def add_edge(self, node, neighbor, cost):
        if node not in self.graph:
            self.graph[node] = []
        self.graph[node].append((neighbor, cost))

    def set_heuristics(self, heuristics):
        self.h_values = heuristics

    def a_star_search(self, start, goal):
        pq = []
        heapq.heappush(pq, (self.h_values[start], start, [start], 0))
        visited = set()

        while pq:
            f_cost, current_node, path, g_cost = heapq.heappop(pq)
            if current_node in visited:
                continue
            visited.add(current_node)

            if current_node == goal:
                print("Path found:", " -> ".join(path))
                print("Total Cost:", g_cost)
                return path, g_cost

            for neighbor, cost in self.graph.get(current_node, []):
                if neighbor not in visited:
                    new_g_cost = g_cost + cost
                    new_f_cost = new_g_cost + self.h_values[neighbor]
                    heapq.heappush(pq, (new_f_cost, neighbor, path + [neighbor], new_g_cost))
```

```
        print("No path found")
        return None, float('inf')

if __name__ == "__main__":
    graph = Graph()
    graph.add_edge("A", "B", 1)
    graph.add_edge("A", "C", 3)
    graph.add_edge("B", "D", 2)
    graph.add_edge("C", "D", 4)
    graph.add_edge("B", "E", 3)
    graph.add_edge("D", "F", 1)
    graph.add_edge("E", "F", 5)

    heuristics = {"A": 6, "B": 4, "C": 4, "D": 2, "E": 3, "F": 0}
    graph.set_heuristics(heuristics)
    start_node, goal_node = "A", "F"
    print("Starting A* Search...")
    graph.a_star_search(start_node, goal_node)
```

6. Explanation of Code

1. **Graph Representation**: Nodes and edges are stored in an adjacency list.
2. **Heuristics**: Defined as a dictionary for each node.
3. **Priority Queue**: Used to prioritize nodes based on their total cost f(n)f(n)f(n).
4. **Main Loop**: Processes nodes, updates costs, and checks if the goal is reached.

7. Graph and Heuristic Example

Graph:

```
   A
  / \
 B   C
/ \   \
D   E   D
 \    /
  F - E
```

Heuristics:-

- A: 6
- B: 4
- C: 4
- D: 2
- E: 3
- F: 0

8. Output Analysis

Name	Type	Size	Value
cost	int	1	4
goal_node	str	1	F
graph	Graph	1	Graph object of __main__ module
heuristics	dict	6	{'A':6, 'B':4, 'C':4, 'D':2, 'E':3, 'F':0}
path	list	4	['A', 'B', 'D', 'F']
start_node	str	1	A

Output:

Path found: A -> B -> D -> F
Total Cost: 4

- The algorithm successfully found the optimal path using the heuristic and edge costs.

*9. Advantages of A Search**

1. Guarantees the shortest path if the heuristic is admissible.
2. Efficient in memory and computation.
3. Combines benefits of BFS and heuristic search.

*10. Applications of A Search in Real World**

- **Navigation Systems**: GPS and maps.
- **Robotics**: Path planning for robots.
- **Game AI**: Solving mazes or finding the optimal moves.

11. Conclusion

The A* Search Algorithm efficiently solves the shortest path problem in graphs using a heuristic function. It balances exploration and cost optimization, making it ideal for real-world applications.